

A. Built-ins — 1-15

1. Indexing

Given:

```
names = ["Alice", "Bob", "Charlie", "David"]
```

Produce:

0 Alice

1 Bob

2 Charlie

3 David

2. Parallel iteration

Given:

```
names = ["Alice", "Bob", "Charlie"]
```

```
scores = [82, 91, 76]
```

Produce:

Alice: 82

Bob: 91

Charlie: 76

3. Every item

Return True if every number in a list is positive.

```
[3, 5, 1, 9] # True
```

```
[3, -1, 5] # False
```

4. At least one

Return True if at least one number is negative.

5. Transform

Turn:

```
["1", "2", "3", "4"]
```

into:

```
[1, 2, 3, 4]
```

6. Filter

From:

```
[1, 8, 3, 10, 5, 12]
```

produce only numbers greater than 5.

7. Sorting by property

Sort:

```
users = [
```

```
{ "name": "Alice", "age": 31 },
```

```
{ "name": "Bob", "age": 22 },
```

```
{ "name": "Charlie", "age": 27 },
```

```
]
```

by age.

8. Find the oldest

Using the same data, return the oldest user.

9. Safe minimum

Write a function that returns the minimum number, but returns None if the input is empty.

10. Sum transformed values

Given:

```
prices = [10, 20, 30]
```

Calculate the total after adding 18% tax.

11. Reverse iteration

Iterate over a list backwards without modifying it.

12. Iterator consumption

Given an iterator, retrieve its next value. Return "done" if it is exhausted.

13. Attribute lookup

Given:

```
class User:
```

```
    name = "Alice"
```

write:

```
    get_value(user, "name")
```

that retrieves the attribute dynamically.

14. Strict parallel iteration

Given two lists that are supposed to have the same length:

```
names = ["Alice", "Bob", "Charlie"]
```

```
scores = [90, 80]
```

iterate over them but raise an error if their lengths differ.

15. Custom sorting

Sort:

```
words = ["Python", "is", "really", "fun"]
```

by length, longest first.

B. collections — 16-30

16. Frequency counter

```
words = ["python", "go", "python", "rust", "go", "python"]
```

Produce the frequency of each word.

17. Most common

Using the same data, find the two most common words.

18. Character frequency

Count the frequency of every character in "mississippi".

19. Group values

Given:

```
people = [  
    ("Alice", "engineering"),  
    ("Bob", "sales"),  
    ("Charlie", "engineering"),  
    ("David", "sales"),  
]
```

Produce:

```
{  
    "engineering": ["Alice", "Charlie"],  
    "sales": ["Bob", "David"]  
}
```

20. Missing keys

Create a dictionary where accessing a missing key automatically creates an empty list.

Then:

```
groups["engineering"].append("Alice")  
should work without initializing "engineering" first.
```

21. Queue

Implement a queue where append-right and remove-left are both efficient.

22. Stack-like operations

Create a structure where you can efficiently add/remove elements from both ends.

23. Sliding window

Given [1, 2, 3, 4, 5], maintain a window of the last 3 values while iterating.

Expected:

```
[1, 2, 3]  
[2, 3, 4]  
[3, 4, 5]
```

24. Group records

Given:

```
records = [  
    ("alice", 10),  
    ("bob", 20),  
    ("alice", 30),  
    ("bob", 40),  
]
```

Group the values by username.

25. Nested counting

Count how many times each (country, city) pair appears.

26. Missing numeric values

Create a dictionary where a missing key starts at 0.

Then count:

```
["apple", "banana", "apple", "apple"]
```

27. Word frequency report

Given a paragraph, produce the 10 most common words.

28. Efficient left popping

You have one million values and repeatedly remove the first item. What standard-library structure would you use? Implement it.

29. Deque rotation

Given [1, 2, 3, 4, 5], rotate it two positions to the right:
[4, 5, 1, 2, 3]

30. Count differences

Given two strings, determine which characters occur more frequently in one than the other.

C. itertools — 31-55

31. Flatten

Turn [[1, 2], [3, 4], [5, 6]] into [1, 2, 3, 4, 5, 6].

32. Chain multiple iterables

Combine [1, 2], [3, 4], and [5, 6] without creating intermediate lists.

33. Cartesian product

Given:

```
colors = ["red", "blue"]
```

```
sizes = ["S", "M", "L"]
```

Produce every possible (color, size) combination.

34. Permutations

Generate every ordering of ["A", "B", "C"].

35. Combinations

Generate every pair from ["A", "B", "C", "D"] without considering order.

36. Combinations with repetition

Generate every pair of numbers from [1, 2, 3] where selecting the same number twice is allowed.

37. Adjacent pairs

Transform [1, 2, 3, 4, 5] into (1,2), (2,3), (3,4), (4,5).

38. Batching

Split range(10) into groups of 3:

```
(0, 1, 2)
```

```
(3, 4, 5)
```

```
(6, 7, 8)
```

```
(9,)
```

39. Take first N

Given an iterator containing millions of items, retrieve only the first 10 without unnecessarily consuming the rest.

40. Skip first N

Given an iterator, ignore its first 100 elements and process everything afterward.

41. Infinite counter

Generate 100, 101, 102, 103, ... without manually incrementing a variable in a loop.

42. Repeat

Produce "hello" five times using an iterator-oriented approach.

43. Accumulation

Given [1, 2, 3, 4], produce 1, 3, 6, 10.

44. Running maximum

Given [3, 1, 8, 2, 10, 4], produce 3, 3, 8, 8, 10, 10.

45. Group consecutive values

Given:

```
["A", "A", "A", "B", "B", "C", "A"]
```

group consecutive equal values. The final "A" is a different group.

46. Split while condition is true

Given [2, 4, 6, 8, 3, 10, 12], take values from the beginning while they remain even.

47. Drop while condition is true

Given [1, 3, 5, 6, 8, 10], skip values while they are odd, then return the remainder.

48. Filter false values

Given [0, 1, False, 2, "", 3, None], return only truthy values.

49. Zip with padding

Given:

```
names = ["Alice", "Bob", "Charlie"]
```

```
scores = [90]
```

Produce:

```
Alice 90
```

```
Bob None
```

```
Charlie None
```

50. Repeated function calls

Call a function repeatedly until some condition is reached.

51. Infinite cycle

Cycle through ["red", "green", "blue"] indefinitely.

52. Flatten one level

Given:

```
[  
  ("a", "b"),  
  ("c", "d"),  
  ("e", "f"),  
]
```

produce a single stream: a b c d e f.

53. Generate all passwords

Given digits = "012", generate every three-digit combination where repetition is allowed.

54. Generate team pairings

Given players = ["A", "B", "C", "D"], generate every unique pair.

55. Lazy pipeline

Build a pipeline that reads numbers, keeps only even numbers, squares them, and takes the first five. Do not create intermediate lists.

D. functools + operator — 56-70

56. Memoization

Create a recursive Fibonacci implementation and cache previously calculated results.

57. Unlimited cache

Write a function whose results are automatically cached indefinitely.

58. Bounded cache

Create a cache that keeps only the most recent 128 results.

59. Partial application

Given:

```
def power(base, exponent):
```

```
...
```

Create `square(x)` and `cube(x)` without defining two new function bodies.

60. Partial configuration

Given:

```
def connect(host, port, timeout):
```

```
...
```

Create a version where `host` and `port` are already fixed.

61. Decorator metadata

Write a decorator that logs calls while preserving the original function's `__name__` and `__doc__`.

62. Reduce

Given `[1, 2, 3, 4]`, calculate `1 * 2 * 3 * 4` using a functional reduction.

63. Reduce — maximum

Find the maximum value using a reduction.

64. Operator function

Sort a list of tuples according to their second element without writing `lambda x: x[1]`.

65. Attribute getter

Sort users by `user.age` using a standard-library callable rather than a lambda.

66. Method caller

Given strings, create a callable that invokes `.upper()`.

67. Function composition

Build `f(g(h(x)))` from three functions.

68. Generic decorator

Write a decorator that works correctly with arbitrary `*args` and `**kwargs` and preserves function metadata.

69. Cache investigation

Write a cached function and determine what happens when called with the same arguments repeatedly. Then inspect its cache statistics.

70. Recursive directory calculation

Create a recursive function that calculates an expensive property of a directory tree and caches repeated calculations.

E. heapq / bisect — 71-80

71. Three largest

Find the three largest numbers in [10, 3, 50, 20, 80, 1, 90] without sorting the entire list.

72. Three smallest

Find the three smallest numbers without sorting the entire list.

73. Priority queue

Create a priority queue where lower numeric priority values are retrieved first. If you want highest-priority-first semantics, adapt the representation accordingly.

74. Running smallest values

Process a stream of numbers while maintaining only the five smallest values seen so far.

75. Sorted insertion

Maintain [1, 3, 5, 8, 10] and insert 6 while keeping it sorted.

76. Find insertion point

Given a sorted list, determine where 7 should be inserted.

77. Range query

Given sorted numbers, find the range of positions where values between 20 and 30 occur.

78. Maintain leaderboard

Maintain a sorted leaderboard as new scores arrive.

79. Merge sorted streams

Given several already-sorted iterables, produce one sorted stream without first combining everything into one giant list.

80. Top-K stream

You receive millions of values from a generator. Return the 20 largest values without storing everything.

F. pathlib, datetime, statistics, math — 81-90

81. Find Python files

Given a directory, recursively find all .py files.

82. File extensions

Given a collection of paths, extract their extensions.

83. File renaming

Rename photo_001.jpg, photo_002.jpg, photo_003.jpg to image_001.jpg, image_002.jpg, image_003.jpg.

84. Directory creation

Create nested directories if they don't already exist.

85. Date difference

Calculate how many days exist between 2026-01-01 and 2026-09-27.

86. Add time

Given a datetime, calculate the time exactly 90 days later.

87. Parse ISO date

Turn "2026-09-27T18:30:00" into a Python datetime.

88. Median

Calculate the median of [10, 2, 8, 4, 100].

89. Mean and standard deviation

Calculate both for [12, 15, 18, 20, 21, 25].

90. Floating-point sum

Calculate the sum of [0.1, 0.1, 0.1, 0.1, 0.1] using a numerically appropriate standard-library tool.

G. contextlib, dataclasses, enum, misc. — 91-100

91. Temporary context

Create a context manager that prints "starting" and "finished" around a block of code.

92. Suppress an expected exception

Attempt to remove a file and quietly ignore the specific exception that occurs when the file doesn't exist.

93. Redirect output

Temporarily redirect stdout into an in-memory buffer.

94. Dataclass

Create User(id, name, email) with generated initialization and useful representation methods.

95. Immutable data

Create a dataclass whose instances cannot be modified after creation.

96. Enum

Represent PENDING, PROCESSING, COMPLETED, and FAILED as an enumeration.

97. Temporary resource

Create a temporary directory, use it, and guarantee cleanup afterward.

98. JSON serialization

Convert `{"name": "Alice", "age": 30}` to JSON and back again.

99. Hashing

Given a file, calculate a cryptographic hash of its contents.

100. The big one

You receive a generator producing millions of log records:

```
{  
  "timestamp": ...,  
  "user": ...,  
  "status": ...,  
  "duration": ...  
}
```

Produce:

- Top 10 users by request count
- Top 10 slowest requests
- Number of requests per status
- Average duration
- Requests grouped by date

Constraints:

- Don't load all records into memory.
- Don't use pandas.
- Use the standard library.
- Prefer iterators/generators where appropriate.
- Think carefully about which data structures are actually necessary.

Reference Map — Problem Shape → Tool to Investigate

This is deliberately a lookup map, not an answer key. Use it after attempting a challenge.

Problem shape	Investigate
Counting things	<code>collections.Counter</code>
Missing dictionary values	<code>collections.defaultdict</code>
Queue/deque operations	<code>collections.deque</code>
Combinations/permutations	<code>itertools</code>
Batching	<code>itertools.batched</code>
Adjacent items	<code>itertools.pairwise</code>
Flattening	<code>itertools.chain</code>
Cartesian combinations	<code>itertools.product</code>
Grouping consecutive values	<code>itertools.groupby</code>
Lazy slicing	<code>itertools.islice</code>
Infinite sequences	<code>itertools.count</code> / <code>cycle</code> / <code>repeat</code>
Cumulative values	<code>itertools.accumulate</code>
Caching	<code>functools.cache</code> / <code>lru_cache</code>
Partially fixing arguments	<code>functools.partial</code>
Decorators	<code>functools.wraps</code>
Operators as functions	<code>operator</code>
Top/bottom K	<code>heapq</code>
Inserting into sorted data	<code>bisect</code>
Paths/files	<code>pathlib</code>
Context managers	<code>contextlib</code>
Structured records	<code>dataclasses</code>
Fixed symbolic values	<code>enum</code>
Dates/time intervals	<code>datetime</code>
Averages/median/stdev	<code>statistics</code>
Mathematical functions	<code>math</code>
JSON	<code>json</code>
Hashes	<code>hashlib</code>
Temporary files	<code>tempfile</code>
Compression	<code>gzip</code> / <code>zipfile</code>
Command-line arguments	<code>argparse</code>
Regular expressions	<code>re</code>

Core Recall Targets